PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

Strings and Collections

Student Details	
Name	B. Lakshmi Sagar
Register Number	192472070
Course	Programming in Java
Assignment	Assignment 2 – CO2

Solutions for Questions 1–7

Note: The GitHub repository link is intentionally omitted. Experimental values are not invented; benchmark programs and result-recording guidance are included where the assignment requires measurements. Run the code on your computer and insert your actual measurements before submission.

Submission note and execution

This report follows the seven questions in the supplied Assignment 2 – CO2 brief. The Java examples use standard Java collections and can be compiled with a recent JDK (Java 8 or later, unless otherwise noted). Save each public class in a file with the same name as the class.

Compile and run: Open a terminal in the folder containing the source file, run `javac FileName.java`, then run `java FileName`. For classes with helper classes, keep them in the same source file unless they are declared public.

The assignment requires repeated timing measurements and raw experimental evidence. The complexity analysis below gives the expected trends, but actual timing and memory readings depend on the computer, JDK, JVM warm-up, and workload. Do not present illustrative values as measurements.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer?

1. Prediction and explanation

String is immutable. Every concatenation creates a new String value; repeated concatenation in a loop repeatedly copies the text already built. StringBuilder is mutable and appends into a growable character buffer, so it normally performs much less copying. StringBuffer provides similar mutable operations but synchronizes its methods, adding overhead.

Approach	Expected behaviour	Best use
String + in loop	Slowest as text grows; many temporary strings and repeated copying.	Small, fixed expressions.
StringBuilder	Usually fastest for one thread; fewer temporary objects.	Building an individual ticket.
StringBuffer	Thread-safe for individual method calls, with synchronization overhead.	A mutable buffer genuinely shared by multiple threads.

2. Three workloads

• Workload A – small: 1 ticket with 6 fields, repeated 100 times.

• Workload B – large: 10,000 tickets, each containing 50 fields.

• Workload C – stress: 25,000 tickets, each containing 100 fields.

• Use identical generated field values for all three approaches. Warm up the JVM, run each test several times, and report the average of at least five measured runs.

3. Java benchmark implementation

```
import java.util.Locale;

public class TicketBenchmark {
    static String[] fields(int count) {
        String[] a = new String[count];
        for (int i = 0; i < count; i++) a[i] = "FIELD-" + i + "-VALUE";
        return a;
    }

    static String withString(String[] fields) {
        String result = "";
        for (String field : fields) result = result + "|" + field;
        return result;
    }

    static String withBuilder(String[] fields) {
        StringBuilder result = new StringBuilder();
        for (String field : fields) result.append('|').append(field);
        return result.toString();
    }

    static String withBuffer(String[] fields) {
        StringBuffer result = new StringBuffer();
        for (String field : fields) result.append('|').append(field);
        return result.toString();
    }
```

```
    static long measure(int mode, String[] data, int tickets) {
        long start = System.nanoTime();
        long checksum = 0;
        for (int i = 0; i < tickets; i++) {
            String s = mode == 0 ? withString(data)
                    : mode == 1 ? withBuilder(data) : withBuffer(data);
            checksum += s.length();
        }
        long elapsed = System.nanoTime() - start;
        if (checksum == 0) System.out.println("No output");
        return elapsed;
    }

    public static void main(String[] args) {
        int[] sizes = {6, 50, 100};
        int[] tickets = {100, 10000, 25000};
        String[] names = {"String +", "StringBuilder", "StringBuffer"};
        for (int w = 0; w < sizes.length; w++) {
            String[] data = fields(sizes[w]);
            System.out.println("\\nWorkload: " + tickets[w] + " tickets, "
                    + sizes[w] + " fields");
            for (int m = 0; m < 3; m++) {
                measure(m, data, 500); // warm-up
                long total = 0;
                for (int r = 0; r < 5; r++)
                    total += measure(m, data, tickets[w]);
                System.out.printf(Locale.US, "%s average: %.3f ms%n",
                        names[m], total / 5.0 / 1_000_000.0);
            }
        }
    }
}
```

4. Interpretation and design decision

The expected result is that StringBuilder scales best for constructing each ticket. StringBuffer is usually slower because the ticket buffer is local to one request and does not need synchronization. For a shared booking log, use a single synchronized logging method or a thread-safe logging facility; if a shared mutable buffer is specifically required, StringBuffer can protect individual operations, but compound operations still need external synchronization. Prefer a dedicated logger/queue for production logging.

Complexity: For n fields with growing total text length, repeated concatenation can copy approximately 1 + 2 + … + n field portions, giving O(n²) character-copy work in the common loop pattern. StringBuilder appends the characters amortized O(n) total. StringJoiner and String.join are also useful for joining known strings; String.format is convenient for formatting but is generally not the most efficient choice in a tight loop.

Space: String concatenation creates intermediate immutable strings; StringBuilder/StringBuffer use a resizable buffer plus the final String returned by toString(). Intermediate object counts depend on compiler/JDK optimizations, so treat them as estimates rather than exact counts.

Q2 – When Is ArrayList an Appropriate Engineering Choice?

1. Complexity comparison

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized at either end
Insert at front	O(n) shifting	O(1) at ends	O(1)
Insert in middle	O(n) search/shift	O(n) to reach position; O(1) link once there	Not designed for indexed middle insertion
Get k-th	O(1)	O(n)	No indexed access
Full iteration	O(n), cache-friendly	O(n), node traversal	O(n)
Space per element	One reference slot plus spare capacity	Node with item and two links	One reference slot plus spare capacity

2. Experimental plan

Test playlist sizes of 1,000, 10,000 and 100,000 songs. For each size, separately time end insertion, front insertion/removal, middle insertion/removal, random get(k), and full iteration. Use the same data, warm up, repeat at least five times, and avoid timing object construction inside the operation being measured unless that is part of the

intended workload.

3. Recommendation

ArrayList is a good choice when the playlist is read or iterated frequently, users jump to arbitrary positions, and most additions occur at the end. Its contiguous backing array gives fast indexed access and good cache locality. I would stop recommending it when frequent insertions/removals near the front or middle dominate, because elements must be shifted. For the changed “Up Next” requirement—frequent additions at both ends and removals from the front—ArrayDeque is more appropriate.

Plain array alternative: A fixed array provides O(1) indexed access and O(n) iteration. Inserting or removing at the front/middle requires manual shifting, O(n); a dynamically resized array has occasional O(n) resize operations and amortized O(1) end append.

Q3 – Engineer the hashCode–equals Contract

1. Why the original class fails

The original Student class defines equality using regNo but inherits Object.hashCode(). Two Student objects with the same registration number can therefore have different hash codes. HashSet first uses hashCode() to select a bucket and then uses equals() among candidates in that bucket. Equal objects with different hash codes may be placed in different buckets, so duplicates can remain.

2. Three tests and predictions

• Test A – two different registration numbers: expected size 2; both are distinct.

• Test B – same registration number but separate objects: with the faulty class, size may be 3 because different identity hash codes can route objects to different buckets; with the corrected class, size is 2.

• Test C – mutate a field used by hashCode after insertion: the object may become difficult to find/remove because it remains in the bucket selected by its old hash. Make equality fields immutable.

3. Corrected complete program

```
import java.util.HashSet;
import java.util.Objects;
import java.util.Set;

class Student {
    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student other = (Student) o;
        return Objects.equals(regNo, other.regNo);
    }

    @Override public int hashCode() {
        return Objects.hash(regNo);
    }

    @Override public String toString() {
        return regNo + " " + name;
    }
}

public class Attendance {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));
        System.out.println("Unique students: " + present.size());
    }
}
```

Expected corrected output: `Unique students: 2`. The exact faulty output is not guaranteed because identity hash codes are implementation-dependent; the key defect is that the class violates the required contract.

4. Alternative structures and complexity

A TreeSet with a Comparator on regNo de-duplicates by comparator equality and takes O(log n) for add/search, with O(n) storage. A HashMap keyed by regNo uses putIfAbsent or containsKey and takes O(1) average lookup/insertion, with O(n) storage. HashSet with a correct contract also provides O(1) average add/contains and O(n) storage.

Q4 – Choosing a Map for a Pharmacy Inventory

1. Map comparison

Map	put/get	Iteration order	Report suitability
HashMap	O(1) average	No guaranteed order	Fast lookup; sort keys separately for code report.
TreeMap	O(log n)	Sorted by key	Naturally produces medicine-code order.
LinkedHashMap	O(1) average	Insertion order	Naturally produces order-added report.

2. Java implementation

```
import java.util.*;

public class PharmacyInventory {
    public static void main(String[] args) {
        Map<String, Integer> stockByCode = new HashMap<>();
        stockByCode.put("MED103", 40);
        stockByCode.put("MED101", 25);
        stockByCode.put("MED102", 60);

        Map<String, Integer> codeOrder = new TreeMap<>(stockByCode);
        Map<String, Integer> insertionOrder = new LinkedHashMap<>();
        insertionOrder.put("MED103", 40);
        insertionOrder.put("MED101", 25);
        insertionOrder.put("MED102", 60);

        System.out.println("Code order: " + codeOrder);
        System.out.println("Added order: " + insertionOrder);
        System.out.println("MED102 stock: " + stockByCode.get("MED102"));
    }
}
```

Expected output: Code order: {MED101=25, MED102=60, MED103=40}; Added order: {MED103=40, MED101=25, MED102=60}; MED102 stock: 60.

3. Experiment and deployment decision

Test 1,000, 10,000 and 100,000 medicines. For each map, measure 100,000 lookups using a pre-generated list of valid keys and separately measure full iteration for both reports. Define an acceptable lookup target before testing—for example, average lookup time below 1 millisecond per lookup on the test machine—and record the actual machine/JDK. HashMap is the natural primary inventory index for frequent code lookups; create a TreeMap view for code-order reporting and retain a LinkedHashMap if insertion-order reporting must be maintained continuously. If both reports are frequent and memory is acceptable, maintain dedicated ordered views or select a primary map based on dominant operations.

Hashtable is generally not preferred in new code because its legacy API synchronizes individual methods, disallows null keys/values, and modern alternatives offer clearer choices such as HashMap, ConcurrentHashMap, or explicit synchronization.

4. Sorted-array alternative

A sorted array supports O(log n) lookup using Arrays.binarySearch and O(n) insertion because elements must be shifted. Producing the code-order report is O(n) because the array is already sorted. It uses O(n) element slots and has low per-entry overhead, but resizing and insertion are costly. HashMap gives O(1) average lookup but no order; TreeMap gives O(log n) lookup/insertion and keeps keys sorted.

Q5 – Derive a Generic Ranking Component from a Principle

1. Design decision

Use a separate Comparator rather than forcing Team to implement Comparable. The required ranking policy may change, and the same generic Leaderboard can then be reused for teams, players, colleges, and quiz teams by supplying a different comparator. The comparison order is points descending, goal difference descending, and name ascending.

2. Completed generic class and comparator

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Team {
    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    @Override public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }

    void add(T item) { items.add(item); }

    List<T> ranking() {
        List<T> copy = new ArrayList<>(items);
        copy.sort(order);
        return copy;
    }

    void addAll(List<? extends T> source) {
        items.addAll(source);
    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> byRank =
            Comparator.comparingInt((Team t) -> t.points).reversed()
                .thenComparing(
                    Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
                .thenComparing(t -> t.name);

        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println(board.ranking());
    }
}
```

Expected output: [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]. Bisons ranks above Panthers because both have 9 points and goal difference 4, so the final alphabetical comparison places “Bisons” before “Panthers”.

3. Correctness, generics and alternatives

The comparator uses lexicographic comparison: the next criterion is considered only when the previous criterion is equal. Each criterion has a consistent total order, so the chain is transitive and deterministic. comparingInt avoids unsafe subtraction, which can overflow. List is useful when accepting a list of objects whose type is T or a subtype of T.

With Comparable, the natural ordering is built into Team and is convenient when one universal order exists. Separate Comparator objects are easier to reuse when different entities or ranking policies need different orders. Sorting a copy costs O(n log n) time and O(n) extra list space. Three stable sorts (name, then goal difference, then points) each cost O(n log n), so total remains O(n log n) but performs roughly three sorts and requires temporary sorting space.

Q6 – Priority, Order and Fairness in a Queue

1. Why PriorityQueue is needed

A normal FIFO Queue serves patients only by arrival order and cannot always select the lowest severity number first. A PriorityQueue orders patients by severity; an increasing arrival counter breaks ties so earlier arrivals with the same severity are served first.

2. Completed program

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {
    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    @Override public String toString() {
        return name + "(S" + severity + ")";
    }
}

public class Triage {
    private long counter = 0;
    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(Comparator
            .comparingInt((Patient p) -> p.severity)
            .thenComparingLong(p -> p.arrival));

    void admit(String name, int severity) {
        queue.add(new Patient(name, severity, counter++));
    }

    Patient next() { return queue.poll(); }

    public static void main(String[] args) {
        Triage er = new Triage();
        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2);

        StringBuilder order = new StringBuilder();
        Patient p;
        while ((p = er.next()) != null)
            order.append(p).append(' ');
        System.out.println(order.toString().trim());
    }
}
```

Expected output: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5). The arrival counter increases for every admitted patient. For equal severity, the smaller counter is treated first.

3. Complexity, display and fairness

PriorityQueue insertion and poll are O(log n); peek is O(1); storage is O(n). Printing the PriorityQueue directly does not guarantee priority order because its iterator exposes the heap's internal arrangement, not a sorted traversal. To display treatment order without changing the live queue, copy it and repeatedly poll the copy.

A sorted LinkedList has O(n) insertion and O(1) removal from the head. Five FIFO queues—one per severity—provide O(1) admission and O(1) selection of the next non-empty severity queue, with O(n) total storage. However, a strict priority policy can starve severity-5 patients if severe cases continually arrive. A fairness rule can promote a waiting

patient by one severity level after a defined waiting interval, while keeping a record of original severity and arrival time.

A city hospital receiving 5,000 patients per day is a reasonable workload for a PriorityQueue in terms of asymptotic cost. Real deployment also requires concurrency control, persistence, monitoring, clinical validation of triage policy, and fairness rules; the data structure alone does not make the system safe.

Q7 – Diagnose, Redesign and Validate

Selected limitation: repeated String concatenation in Q1

A. Diagnosis: The original ticket generator uses immutable String concatenation inside a loop. As the ticket grows, previous characters are repeatedly copied into new String objects. The amount of copying grows approximately quadratically with the number of appended fields. This is unnecessary repeated work and creates temporary objects.

B. Design objective: For a ticket with n fields, reduce character-copy work from $O(n^2)$ to amortized O(n), while preserving the exact ticket contents and field order. Compare both implementations with identical input.

C. Revised design: Replace the loop's String + operation with a local StringBuilder. Append each delimiter and field, then call toString() once. A local builder avoids synchronization because it is not shared between threads.

```
// Original
String ticket = "";
for (String field : fields) {
    ticket = ticket + "|" + field;
}

// Redesigned
StringBuilder ticketBuilder = new StringBuilder();
for (String field : fields) {
    ticketBuilder.append('|').append(field);
}
String ticket = ticketBuilder.toString();
```

Pseudocode: create an empty mutable buffer → for each field append delimiter and field → convert buffer to String once → return ticket. Extra space is O(total ticket length); StringBuilder may reserve spare capacity, but it avoids retaining every intermediate immutable result.

D. Prediction: For a small ticket, the difference may be negligible. For many tickets with many fields, StringBuilder should reduce average construction time and temporary allocation substantially.

E. Validation method: Run the Q1 benchmark with the same three workloads and at least five measured repetitions after warm-up. Confirm that both methods produce identical output by comparing the generated strings. Record the actual timings and JVM memory observations below.

Workload	String + average (ms)	StringBuilder average (ms)	Improvement
100 tickets × 6 fields	Record locally	Record locally	Calculate locally
10,000 × 50 fields	Record locally	Record locally	Calculate locally
25,000 × 100 fields	Record locally	Record locally	Calculate locally

Improvement percentage = ((String time – StringBuilder time) / String time) × 100. Memory readings from Runtime.totalMemory() – Runtime.freeMemory() are approximate and affected by garbage collection; use the same procedure for both runs and do not treat one reading as exact allocation.

F. Final defence: StringBuilder is the preferred per-ticket construction tool because it reduces repeated copying and is not burdened by synchronization. String concatenation remains acceptable for a small fixed number of fields because it is simpler and the performance difference may not matter. The redesign meets the algorithmic objective; the magnitude of improvement must be filled in from the student's own run rather than fabricated.

Final checklist

• Replace every 'Record locally' cell with measurements from your own execution environment.

• Run every Java program, verify the displayed output, and capture your own output screenshots.

• Keep source files and raw benchmark results in separate task folders in your GitHub repository.

• Add your Java version and compile/run instructions to README.md. The GitHub link is omitted from this PDF as requested.

• Be prepared to explain the predictions, code, complexity, measurements and trade-offs during the verification discussion.